Tab 1

OTW — ON THE WATER

Master Project Management Plan

Project Type: Collaborative Web Application
Project Concept: Creative and interactive gala/trip planner for circles of friends
Frontend: Nextjs+ TypeScript + Shadcn + Tanstack query
Backend: Java 21 + Spring Boot
Database: Supabase PostgreSQL
Authentication: Supabase Auth
Caching: Redis
Realtime: Supabase Realtime
Photo Storage: Google Drive
Map: Leaflet + OpenStreetMap
Frontend Deployment: Vercel
Backend Deployment: Persistent Java/Spring hosting/Render
Project Approach: Agile / Sprint-based
Target: Mobile-first, secure, responsive, scalable, fun, and easy to use

---

1. PROJECT VISION

OTW, or **On The Water**, is a collaborative planning platform where barkadas, friend groups, couples, and small groups can create a shared gala room and plan an outing together.

The goal is not to make a traditional itinerary application.

The goal is to make planning a gala feel like part of the gala itself.

Instead of:

"Guys, ano gagawin natin Saturday?"

OTW should turn the planning process into an interactive experience.

The application should allow users to:

- create or join a gala
- see who is actually coming
- collaboratively build an itinerary
- assign items to bring
- estimate the trip budget
- split expenses
- see who is currently on the way
- optionally share live location
- upload memories
- use playful decision-making features
- plan dates/couples trips through a different mode

The core principle is:

Useful first, fun second, beautiful third, but all three should work together.

---

2. PROJECT OBJECTIVES

Primary Objectives

O1 — Simplify group gala planning

A user should be able to create a gala and invite friends with minimal friction.

O2 — Centralize planning

The following should exist in one room:

- attendees
- itinerary
- checklist
- budget
- expenses
- location
- memories

O3 — Make planning collaborative

Changes made by one person should be reflected to other participants where appropriate.

O4 — Make the experience playful

OTW should have witty interactions and branded terminology without sacrificing usability.

O5 — Protect private room information

A person must not be able to access another group's room simply by guessing or manipulating URLs/API parameters.

O6 — Remain responsive under concurrent use

The system should minimize unnecessary database requests and use caching, realtime events, pagination, rate limiting, and efficient queries.

O7 — Remain manageable for a student project

The architecture should be modular but should not unnecessarily become a microservices system.

---

3. PROJECT SUCCESS CRITERIA

The project is successful when:

Area	Success Condition
Accessibility	User can create/join a gala without confusing steps
Usability	Core tasks can be completed on mobile without zooming
Collaboration	Important room changes appear without manual page refresh
Security	Users cannot access unauthorized rooms/data
Performance	Common read requests remain responsive under concurrent users
Reliability	Temporary cache failure does not destroy core functionality
Data Integrity	Money, attendance, and room data remain consistent
Maintainability	Features are separated and documented
Deployment	Production environment works outside the developers' computers
Presentation	Team can clearly explain architecture, workflow, security, and testing

4. PROJECT SCOPE

4.1 MVP — MUST HAVE

These features are considered essential.

Gala Management

- Create gala
- Join gala
- Room code
- Nickname
- Gala title
- Schedule
- Meeting location
- Room membership

Collaboration

- RSVP
- Participant list
- Itinerary
- Shared checklist

Money

- Budget estimate
- Expenses
- KKB splitting
- Settlement calculation

Realtime

- RSVP updates
- itinerary changes
- checklist updates
- participant updates

Security

- authentication
- room authorization
- role authorization
- validation
- rate limiting
- secure API

Performance

- Redis cache
- database indexing
- pagination
- efficient queries
- load testing

---

5. STRETCH FEATURES

These are developed only after MVP functionality is stable.

- Couple Mode
- Date decision randomizer
- “Sino Mag-aabono?” wheel
- Live GPS

- OTW status meter
- Google Drive photo vault
- more elaborate animations
- advanced notifications
- additional themes

This prevents a common student-project problem:

80% UI, 40% functionality, 0% testing.

---

6. OUT OF SCOPE FOR VERSION 1

Do not accidentally turn OTW into a giant social network.

Do not include initially:

- public social feed
- public profiles
- direct messaging
- public friend system
- payment processing
- actual GCash transactions
- hotel booking
- restaurant reservation API
- transportation booking
- navigation turn-by-turn system
- AI itinerary generation unless explicitly added later

---

7. PROJECT TEAM STRUCTURE

For a student team, divide responsibilities by ownership rather than having everyone edit everything.

Suggested Roles

Project Manager / Product Owner

Responsible for:

- requirements
- task tracking
- scope control
- prioritization
- documentation
- acceptance testing
- deadlines

Frontend Lead

Responsible for:

- React
- UI implementation
- responsive behavior
- interactions
- frontend state
- realtime UI

Backend Lead

Responsible for:

- Spring Boot
- API
- authorization
- business logic

- validation
- database integration
- Redis

Database / QA / DevOps

Responsible for:

- Supabase migrations
- indexes
- RLS
- test strategy
- testing
- deployment
- performance testing
- security testing

One person can have multiple roles.

---

8. PROJECT GOVERNANCE

Use these statuses in your project board:

- BACKLOG
- ↓
- READY
- ↓
- IN PROGRESS
- ↓
- CODE REVIEW / REVIEW
- ↓
- QA
- ↓

- UAT
- ↓
- DONE

Additional status:

- BLOCKED

Never mark something DONE simply because the code runs.

It becomes DONE only when:

1. requirement is satisfied
2. expected behavior works
3. errors are handled
4. security is checked
5. responsive behavior is checked
6. relevant tests pass
7. documentation is updated when necessary

---

9. PROJECT ARTIFACTS

Before coding heavily, create these documents.

- docs/
- ├── PROJECT_CHARTER.md
- ├── REQUIREMENTS.md
- ├── SCOPE.md
- ├── USER_STORIES.md
- ├── USER_FLOWS.md

- ├── UI_UX_REQUIREMENTS.md
- ├── BRAND_GUIDE.md
- ├── ARCHITECTURE.md
- ├── DATABASE_DESIGN.md
- ├── API_SPEC.md
- ├── REALTIME_SPEC.md
- ├── CACHE_STRATEGY.md
- ├── SECURITY_PLAN.md
- ├── TEST_PLAN.md
- ├── UAT_PLAN.md
- ├── DEPLOYMENT_PLAN.md
- ├── RISK_REGISTER.md
- ├── CHANGE_LOG.md
- └── PERFORMANCE_PLAN.md

These documents are the project's **source of truth**.

Claude should read them before working.

---

10. MASTER PROJECT PHASES

PHASE 0 — Project Initiation

Goal:

Define exactly what OTW is before development.

Deliverables:

- Project Charter
- Scope

- stakeholder definition
- project goals
- MVP boundary
- success criteria
- initial risks

PHASE 1 — Product Discovery

Goal:

Define how people will actually use OTW.

Deliverables:

- personas
- user journeys
- user stories
- functional requirements
- non-functional requirements
- edge cases

PHASE 2 — UX + Branding

Goal:

Create the visual and interaction direction.

Deliverables:

- brand identity
- logo direction
- color system

- typography
- design principles
- wireframes
- user flows
- mobile layouts

Do not fully polish the UI yet.

PHASE 3 — Technical Planning

Goal:

Freeze the architecture before major development.

Deliverables:

- architecture diagram
- database ERD
- API contract
- authentication flow
- authorization model
- realtime architecture
- caching architecture
- deployment architecture

PHASE 4 — Foundation

Goal:

Make sure the project skeleton is stable.

Deliverables:

- repository
- branch strategy
- environments
- Next.js foundation
- Spring Boot foundation
- Supabase
- authentication
- Redis
- CI checks

PHASE 5 — Core MVP

Build in this order:

- Room
- ↓
- Members
- ↓
- RSVP
- ↓
- Dashboard
- ↓
- Itinerary
- ↓
- Checklist
- ↓
- Budget
- ↓
- Expenses

PHASE 6 — Realtime + Performance

Then add:

- Realtime
- Redis
- rate limiting
- query optimization
- pagination
- concurrency handling

PHASE 7 — Fun Layer

Only after the core works:

- Wheel
- Couple Mode
- Decision Maker
- OTW Meter
- GPS
- Photo Vault

PHASE 8 — QA + Security

Perform:

- functional testing
- integration testing

- security testing
- usability testing
- mobile testing
- performance testing
- load testing
- UAT

PHASE 9 — Release

- production environment
- monitoring
- deployment
- smoke testing
- documentation
- presentation/demo preparation

---

11. DETAILED WORK BREAKDOWN STRUCTURE

WBS 0 — Project Initiation

0.1 Project Charter

Output: `PROJECT_CHARTER.md`

Contains:

- project name

- problem
- solution
- target users
- objectives
- scope
- stakeholders
- constraints
- success criteria

Claude Prompt

- Act as OTW's project manager.
-
- Using CLAUDE.md and existing project files, create docs/PROJECT_CHARTER.md.
-
- Define:
- problem, target users, solution, objectives, constraints, scope boundaries, stakeholders, assumptions, dependencies, risks, and measurable success criteria.
-
- Do not write application code.
- Keep decisions specific to OTW.
- Flag unclear requirements instead of inventing them.

0.2 MVP Definition

Output: `SCOPE.md`

Classify every feature:

- MUST HAVE

- SHOULD HAVE
- COULD HAVE
- OUT OF SCOPE

Claude Prompt

- Act as a product manager.
-
- Review OTW's current feature list and classify each feature as MUST, SHOULD, COULD, or OUT OF SCOPE for V1.
-
- Prioritize by user value, implementation risk, security impact, and project time.
-
- Return:
- feature matrix + rationale + recommended MVP boundary.
-
- Do not code.

0.3 Stakeholder / User Definition

Create:

- Host
- Member
- Couple user
- Future admin

Claude Prompt

- Define OTW's user roles and responsibilities.

-
- For each role specify:
- goals, actions, permissions, frustrations, risks, and success criteria.
-
- Output docs/USER_ROLES.md.
-
- Do not design UI or code.

---

12. WBS 1 — REQUIREMENTS

1.1 User Stories

Example:

- As a gala host,
- I want to create a room,
- so my friends can join one shared planning space.

Create stories for every major feature.

Claude Prompt

- Convert OTW requirements into user stories.
-
- Format:
- ID
- Role
- Story
- Acceptance Criteria

- Priority
- Dependencies
- Edge Cases
-
- Cover room, members, RSVP, itinerary, checklist, budget, expenses, realtime, GPS, photos, security, and performance.
-
- Output docs/USER_STORIES.md.
- Do not code.

1.2 Acceptance Criteria

Every feature must have a “Done means…” definition.

Example:

RSVP

- Given I am a room member
- When I choose GOING
- Then my RSVP becomes GOING
- And authorized members receive the change
- And refreshing preserves the state.

Claude Prompt

- Create measurable acceptance criteria for every OTW MVP feature.
-
- Use Given/When/Then where useful.
- Include:

- happy path
- validation
- authorization
- failure path
- concurrency case
- mobile behavior
-
- Do not write implementation code.

1.3 Edge Case Inventory

This is something student teams usually skip.

Examples:

- What if room code does not exist?
- What if two people join simultaneously?
- What if host leaves?
- What if RSVP changes while budget is loading?
- What if Redis is unavailable?
- What if database is temporarily unavailable?
- What if photo upload fails halfway?
- What if GPS permission is denied?
- What if two people edit the same itinerary?

Claude Prompt

- Perform OTW edge-case analysis.
-
- For each core feature identify:

- invalid input
- empty state
- concurrent action
- network failure
- authorization failure
- duplicate request
- expired session
- third-party failure
-
- Return a prioritized edge-case matrix.
- Do not code.

---

13. WBS 2 — USER FLOW DESIGN

Create actual flows before UI.

Flow A — Create Gala

- Landing
- ↓
- Plan a Gala
- ↓
- Nickname
- ↓
- Gala details
- ↓
- Meeting location
- ↓
- Create

- ↓
- Room created
- ↓
- Share code
- ↓
- Dashboard

Flow B — Join Gala

- Landing
- ↓
- Join Gala
- ↓
- Room code
- ↓
- Nickname
- ↓
- Validate
- ↓
- Join
- ↓
- Dashboard

Flow C — Expense

- Dashboard
- ↓
- Budget
- ↓
- Add Expense

- ↓
- Amount
- ↓
- Payer
- ↓
- Participants
- ↓
- Save
- ↓
- Settlement

Claude Prompt

- Create detailed UX flows for OTW.
-
- Cover:
- create gala
- join gala
- RSVP
- itinerary
- checklist
- budget
- expense splitting
- GPS
- photos
- fun features
-
- For each flow include:
- entry point
- steps
- decision points
- success state
- error state

- empty state
- exit point
-
- Output docs/USER_FLOWS.md.
- No code.

---

14. WBS 3 — UI/UX PLANNING

Since you currently have **no branding**, this should be an actual project phase.

3.1 Brand Discovery

Decide:

- OTW visual personality
- logo direction
- typography
- color palette
- imagery
- tone of voice
- witty terminology
- animation philosophy

The brand should communicate:

gala + friendship + spontaneity + travel + playful chaos

3.2 Design Principles

Example:

- Mobile first
- Few steps
- Large touch targets
- Information hierarchy
- Playful microcopy
- Smooth transitions
- No clutter
- Fast perceived performance
- Accessible contrast
- Clear feedback

Claude Prompt

- Act as OTW's UX strategist and brand planner.
-
- Develop 3 distinct design directions for OTW.
-
- For each define:
- personality
- color direction
- typography direction
- UI shape language
- motion style
- illustration style
- microcopy tone
- brand keywords
-
- Evaluate each against:
- friend-group usability
- mobile usability
- memorability

- technical feasibility
-
- Do not choose a final direction for us.
- Do not code.

3.3 Wireframe Planning

Required screens:

- Landing
- Create Gala
- Join Gala
- Room Dashboard
- Overview
- Members
- Itinerary
- Checklist
- Budget
- Expenses
- Map
- Photos
- Fun Mode
- Settings
- Error pages

Claude Prompt

- Create a low-fidelity wireframe specification for OTW.
-
- For each screen define:

- purpose
- primary action
- secondary actions
- content hierarchy
- components
- mobile layout
- desktop layout
- empty state
- loading state
- error state
-
- Do not specify final colors or write code.

---

15. WBS 4 — SYSTEM ARCHITECTURE

This is where your stack becomes official.

Architecture decision

- Next.js
- ↓
- Spring Boot API
- ↓
- Supabase PostgreSQL
-
- ↘ Redis
-

- Supabase Realtime
- ↓
- Connected room members
-
- Google Drive
- ↓
- Original photos

Supabase currently documents Realtime Broadcast as the recommended approach over Postgres Changes when scalability and security matter, and private channels can be protected through RLS authorization.

Supabase also supports anonymous authenticated users, which matches OTW's low-friction nickname-based onboarding, although the user remains device/session dependent unless they later link another authentication method.

4.1 Architecture Decision Record

Create:

- docs/ARCHITECTURE.md

Include:

- frontend responsibilities
- backend responsibilities
- database responsibilities
- Redis responsibilities
- Realtime responsibilities
- Google Drive responsibilities
- deployment responsibilities

- security boundaries

Claude Prompt

- Act as a senior software architect.
-
- Design OTW's production architecture using:
- Next.js/React
- Spring Boot
- Supabase PostgreSQL/Auth/Realtime
- Redis
- Google Drive
-
- For each component define:
- responsibility
- data flow
- security boundary
- failure behavior
- scaling concern
- reason it was selected
-
- Also identify architectural anti-patterns we must avoid.
-
- Do not code.
- Output docs/ARCHITECTURE.md.

---

16. WBS 5 — DATABASE PLANNING

Before implementation, finalize entities.

Core:

- rooms
- room_members
- itinerary_items
- checklist_items
- expenses
- expense_shares
- photos

Then define:

- relationships
- constraints
- indexes
- ownership
- deletion behavior
- timestamps
- concurrency fields

Claude Prompt

- Create OTW's database design.
-
- For every table define:
- purpose
- fields
- types
- PK
- FK
- unique constraints
- indexes
- relationships
- deletion behavior

- ownership/security considerations
-
- Then produce a Mermaid ERD.
-
- Do not generate application code.
- Output docs/DATABASE_DESIGN.md.

---

17. WBS 6 — API PLANNING

Do this before frontend-backend integration.

Define:

- endpoint
- method
- purpose
- request
- response
- auth
- role
- validation
- errors
- cache behavior
- realtime event

Claude Prompt

- Create OTW's REST API contract.
-
- For every endpoint define:

- HTTP method
- path
- purpose
- authentication
- authorization
- request schema
- response schema
- validation
- error codes
- cache behavior
- realtime side effect
-
- Use /api/v1.
-
- Do not implement the endpoints.
- Output docs/API_SPEC.md.

---

18. WBS 7 — AUTHORIZATION MODEL

This should be planned separately from login.

Authentication asks:

> Who are you?

Authorization asks:

> Are you allowed to access this room?

Supabase Auth uses JWT-based authentication and integrates with database authorization through RLS.

For OTW:

- Authenticated user
- ↓
- User identity
- ↓
- Room membership
- ↓
- Role
- ↓
- Allowed action

Example:

- HOST
- ├── create room
- ├── edit room
- ├── remove member
- └── manage room
-
- MEMBER
- ├── RSVP
- ├── itinerary
- ├── checklist
- ├── expenses
- └── location sharing

Claude Prompt

- Design OTW's authorization matrix.

-
- Rows:
- room actions
- member actions
- itinerary actions
- checklist actions
- expense actions
- photo actions
- GPS actions
-
- Columns:
- HOST
- MEMBER
- UNAUTHORIZED
-
- For every action define:
- allowed?
- conditions?
- ownership rule?
- security risk?
-
- Output docs/AUTHORIZATION_MATRIX.md.
- No code.

---

19. WBS 8 — CACHING & PERFORMANCE PLAN

This gets its own project phase.

Your system should have:

- NEXT.JS / CLIENT CACHE
- ↓
- SPRING BOOT
- ↓
- REDIS
- ↓
- SUPABASE POSTGRES

Spring Boot officially supports cache abstraction and Redis as a cache provider.

Cache candidates

- room overview
- member list
- itinerary
- checklist
- budget summary
- photo metadata

Do not cache blindly.

The PM task is to identify:

- what is frequently read
- what changes frequently
- what can tolerate slightly stale data
- what must always be authoritative

Performance requirement

Use these as **initial engineering targets**, not promises:

- P95 normal API response < 500 ms
- P99 normal API response < 1.5 s
- Error rate < 1%
- No unbounded collection endpoints
- No obvious N+1 queries

Actual targets should be refined using your load-test results.

Claude Prompt

- Act as OTW's performance architect.
-
- Design a cache/performance strategy.
-
- For each major endpoint classify:
- cacheable?
- TTL
- key
- invalidation event
- acceptable staleness
- fallback if Redis fails
-
- Also identify:
- DB indexes
- pagination requirements
- N+1 risks
- large payload risks
- frontend duplicate-fetch risks
- Realtime vs polling opportunities
-
- Output docs/CACHE_STRATEGY.md and docs/PERFORMANCE_PLAN.md.
-

- Do not code.

---

20. WBS 9 — REALTIME PLAN

Supabase Realtime can provide low-latency Broadcast messaging, and its private channel authorization can be tied to authenticated users and room membership.

Define events before implementation.

- member.joined
- member.left
- member.rsvp.changed
-
- itinerary.created
- itinerary.updated
- itinerary.completed
-
- checklist.created
- checklist.updated
-
- expense.created
-
- location.updated
-
- photo.uploaded

The important PM decision:

Realtime should broadcast changes, not constantly replace the database.

Claude Prompt

- Design OTW's realtime event contract.
-
- Define:
- event name
- trigger
- topic
- sender
- authorized receivers
- payload
- UI reaction
- reconnection behavior
- duplicate-event handling
-
- Use private room channels.
-
- Output docs/REALTIME_SPEC.md.
- No code.

---

21. WBS 10 — SECURITY MANAGEMENT PLAN

Create a threat inventory.

Threats

- Room-code guessing
- Unauthorized room access

- Broken object authorization
- Privilege escalation
- JWT misuse
- XSS
- CSRF where applicable
- malicious file uploads
- API abuse
- rate-limit bypass
- credential exposure
- location privacy issues
- cache data leakage

OWASP's API Security Top 10 specifically highlights risks such as broken object-level authorization, broken function-level authorization, authentication failures, and unrestricted resource consumption. (api-security.owasp.org)

Claude Prompt

- Act as OTW's application security reviewer.
-
- Create a threat model for:
- authentication
- room access
- membership
- API objects
- expenses
- photos
- GPS
- Realtime
- Redis
- third-party integrations
-
- For each threat define:

- attack
- impact
- likelihood
- mitigation
- verification method
- owner
-
- Output docs/SECURITY_PLAN.md.
-
- Do not code.

---

22. WBS 11 — DEVELOPMENT MANAGEMENT

Only after the planning documents are approved should implementation begin.

Use this feature sequence:

- Foundation
- ↓
- Authentication
- ↓
- Room
- ↓
- Membership
- ↓
- Dashboard
- ↓
- RSVP

- ↓
- Itinerary
- ↓
- Checklist
- ↓
- Budget
- ↓
- Expenses
- ↓
- Realtime
- ↓
- Redis
- ↓
- Fun features
- ↓
- GPS
- ↓
- Photos

Do not develop all features simultaneously.

---

23. SPRINT STRUCTURE

SPRINT 0 — Discovery

Duration: 2–3 days

Deliver:

- Project Charter

- Scope
- MVP
- User stories
- User flows
- Risk register

Sprint Exit Gate

The team can answer:

> What exactly are we building?

If not, don't start major implementation.

---

24. SPRINT 1 — UX + Architecture

Deliver:

- brand direction
- wireframes
- design system plan
- architecture
- ERD
- API contract
- security model
- cache strategy
- realtime strategy

Sprint Exit Gate

The team can answer:

> How will the system work before we start implementing it?

---

25. SPRINT 2 — Foundation + Room

Deliver:

- repository
- environments
- authentication
- create gala
- join gala
- room membership
- dashboard shell

Sprint Exit Gate

A user can:

- open OTW
- ↓
- create gala
- ↓
- get room code
- ↓
- another user joins
- ↓
- both see the same room

---

26. SPRINT 3 — Collaboration

Deliver:

- RSVP
- itinerary
- checklist
- member states
- Realtime

Sprint Exit Gate

Two different devices can collaborate successfully.

---

27. SPRINT 4 — Money

Deliver:

- budget
- expense entry
- expense sharing
- settlement

Sprint Exit Gate

The financial calculations are correct for tested scenarios.

---

28. SPRINT 5 — Performance + Advanced Features

Deliver:

- Redis
- rate limiting
- map
- GPS
- photo system
- fun features

Important:

This sprint begins only when core MVP is stable.

---

29. SPRINT 6 — QA + Security + Release

Deliver:

- functional QA
- mobile QA
- security testing
- load testing
- bug fixing

- production deployment
- UAT

---

30. GATE SYSTEM

Use formal project gates.

GATE A — Requirements Approved

No major unknown MVP feature.

GATE B — Design Approved

User flows and UI direction finalized.

GATE C — Architecture Approved

Database, API, security, cache, realtime decisions documented.

GATE D — MVP Complete

All core user journeys work.

GATE E — Security Passed

No critical security issue remains open.

GATE F — Performance Passed

Load test results documented.

GATE G — UAT Passed

Actual testers successfully complete defined scenarios.

GATE H — Release

Production environment validated.

---

31. RISK REGISTER

Maintain this from Day 1.

Risk	Pr	Mitigation
Scope becomes too large	Hi	Freeze MVP

No finalized UI	Hi	Finish UX before polishing
Realtime complexity	M	Design event contract first
DB becomes slow	M	indexes + pagination + Redis
Too many requests	M	cache + client caching + realtime

Cache contains stale data	M	explicit invalidation
Redis outage	L	DB fallback
Unauthorized room access	M	authorization matrix + tests
Google Drive failure	M	upload retry + metadata states

GPS privacy problem	L	explicit opt-in + room authorization
Mobile layout breaks	Hi	mobile-first QA
Deployment failure	M	staging environment
Claude generates unwanted architecture	M	CLAUDE.md + review gates

Team conflicts in Git	M	branch rules
Delayed development	M	feature freeze / stretch removal

---

32. CHANGE MANAGEMENT

Once requirements are approved, don't casually add features.

Any new feature gets:

- Change Request
- ↓
- Reason
- ↓
- Value
- ↓
- Estimated effort
- ↓
- Dependency impact
- ↓

- Security impact
- ↓
- Performance impact
- ↓
- Decision

Allowed decisions:

- APPROVE
- DEFER
- REJECT

This protects you from:

> "Uy lagyan din natin ng chat."

then:

> "Uy lagyan din natin ng booking."

then:

> "Uy gumawa rin tayo ng social media."

---

33. CLAUDE AI MANAGEMENT SYSTEM

This is where Claude becomes useful without letting Claude become the project manager.

You control the project.

Claude assists with:

- planning
- analysis
- documentation
- architecture review
- requirements
- risk analysis
- test planning
- QA
- code review
- debugging
- implementation

The important rule:

Never let Claude decide scope on its own.

---

34. MASTER CLAUDE PROJECT PROMPT

Put this in `CLAUDE.md`.

- PROJECT: OTW — On The Water
-
- OTW is a collaborative gala/trip planner for friend groups and couples.
-
- STACK:
- Next.js + React + TypeScript
- Spring Boot + Java 21

- Supabase PostgreSQL/Auth/Realtime
- Redis
- Google Drive
- Leaflet/OpenStreetMap
-
- PROJECT RULE:
- This is a managed software project. Requirements, architecture, scope, security, and acceptance criteria are defined in /docs and must be treated as the source of truth.
-
- YOUR ROLE:
- Depending on the task, act as:
- Product Manager
- Business Analyst
- UX Strategist
- Software Architect
- QA Engineer
- Security Reviewer
- Performance Reviewer
- Developer
-
- DEFAULT BEHAVIOR:
- 1. Inspect relevant existing docs/files first.
- 2. Do not invent missing requirements silently.
- 3. Do not expand scope.
- 4. Do not code unless the task explicitly requests implementation.
- 5. When planning, produce decisions, assumptions, dependencies, risks, acceptance criteria, and deliverables.
- 6. Prefer simple solutions appropriate for a student project.
- 7. Preserve the documented architecture.
- 8. Flag architecture conflicts before modifying anything.
- 9. Security and authorization are mandatory.
- 10. Performance and concurrency must be considered for shared data.
- 11. PostgreSQL is the source of truth.

- 12. Redis is a performance/ephemeral-data layer, not the source of truth.
- 13. Private room data must never be exposed across rooms.
- 14. Realtime must use authorized private channels.
- 15. Money must use exact decimal arithmetic.
- 16. Do not introduce microservices unless explicitly approved.
- 17. Do not add dependencies without justification.
- 18. Do not perform large rewrites when a targeted change is sufficient.
-
- DOCUMENT PRIORITY:
- 1. PROJECT_CHARTER
- 2. SCOPE
- 3. REQUIREMENTS
- 4. USER_STORIES
- 5. USER_FLOWS
- 6. ARCHITECTURE
- 7. DATABASE_DESIGN
- 8. API_SPEC
- 9. SECURITY_PLAN
- 10. CACHE_STRATEGY
- 11. TEST_PLAN
-
- When these conflict, report the conflict instead of silently choosing.
-
- FINAL RESPONSE FORMAT:
- TASK STATUS
- DECISIONS
- FILES/DOCUMENTS CHANGED
- RISKS
- DEPENDENCIES
- NEXT ACTION

---

35. CLAUDE PROMPT RULE

Every task prompt should be **small**.

Do not send Claude a 2,000-word prompt every time.

Because `CLAUDE.md` already contains the permanent context.

Use:

- Follow CLAUDE.md.
-
- Task:
- ...
-
- Deliverable:
- ...
-
- Constraints:
- ...
-
- Do not code.

That is enough.

---

36. HIGH-VALUE CLAUDE PROMPTS PER PROJECT TASK

PM-01 — Project Audit

- Follow CLAUDE.md.
-
- Audit the current OTW project.
-
- Review docs, repo structure, implemented features, and unresolved issues.
-
- Return:
- completed
- in-progress
- missing
- blocked
- scope risks
- architecture risks
- recommended next task.
-
- Do not modify code.

PM-02 — Requirements Review

- Follow CLAUDE.md.
-
- Review OTW requirements for ambiguity, duplication, contradiction, and missing acceptance criteria.
-
- Return only:
- issue
- why it matters
- recommended clarification
- priority.
-
- Do not rewrite unrelated requirements.

PM-03 — Feature Prioritization

- Follow CLAUDE.md.
-
- Prioritize OTW features by:
- user value
- project risk
- dependency
- development effort
- security impact
-
- Classify:
- MUST
- SHOULD
- COULD
- OUT.
-
- Do not code.

PM-04 — Sprint Planning

- Follow CLAUDE.md.
-
- Create the next OTW sprint plan.
-
- Include:
- goal

- tasks
- dependencies
- deliverables
- acceptance criteria
- risks
- definition of done
-
- Keep the sprint realistic for a student team.
- Do not add new features.

PM-05 — Task Breakdown

- Follow CLAUDE.md.
-
- Break this feature into implementation-ready tasks:
-
- [FEATURE]
-
- For each task provide:
- objective
- owner type
- dependency
- input
- output
- acceptance criteria
- risk.
-
- No code.

PM-06 — Architecture Review

- Follow CLAUDE.md.
-
- Review the proposed solution for [FEATURE].
-
- Check:
- security
- scalability
- performance
- maintainability
- database impact
- cache impact
- Realtime impact
-
- Return:
- approved
- concerns
- required changes
- optional improvements.
-
- No code.

PM-07 — Database Review

- Follow CLAUDE.md.
-
- Review the database design for [FEATURE].

-
- Check:
- relationships
- constraints
- indexes
- query patterns
- N+1 risk
- concurrency
- data ownership
- privacy
-
- Return findings by severity.
- No code.

PM-08 — API Contract Review

- Follow CLAUDE.md.
-
- Review the API design for [FEATURE].
-
- Check:
- request validation
- authorization
- response consistency
- error handling
- pagination
- rate limiting
- cache behavior
- Realtime side effects
-
- Return required changes only.

- No implementation.

PM-09 — UX Review

- Follow CLAUDE.md.
-
- Review the UX flow for [FEATURE].
-
- Check:
- number of steps
- clarity
- mobile usability
- empty states
- loading states
- errors
- accessibility
- user feedback
-
- Return improvement recommendations without redesigning unrelated screens.

PM-10 — Security Review

- Follow CLAUDE.md.
-
- Perform a focused security review for [FEATURE].
-
- Check:

- authentication
- authorization
- object ownership
- input validation
- rate limiting
- privacy
- secret exposure
- Realtime permissions
- cache isolation
-
- Rank findings:
- CRITICAL
- HIGH
- MEDIUM
- LOW
-
- Do not code fixes yet.

PM-11 — Cache Review

- Follow CLAUDE.md.
-
- Review caching for [FEATURE].
-
- Determine:
- what should be cached
- TTL
- key
- invalidation trigger
- staleness tolerance
- authorization requirement

- DB fallback
-
- Flag any cache design that could leak private room data.
- No code.

PM-12 — Performance Review

- Follow CLAUDE.md.
-
- Review [FEATURE] for performance risks.
-
- Check:
- DB queries
- payload size
- frontend requests
- cache usage
- Realtime
- rendering
- concurrency
- third-party API calls
-
- Return:
- risk
- expected impact
- recommended action
- measurement method.

PM-13 — QA Planning

- Follow CLAUDE.md.
-
- Create QA test cases for [FEATURE].
-
- Cover:
- happy path
- validation
- authorization
- edge cases
- network failure
- concurrency
- mobile
- Realtime
- cache behavior
-
- Use test IDs.
- No code.

PM-14 — UAT Planning

- Follow CLAUDE.md.
-
- Create UAT scenarios for OTW [FEATURE].
-
- Write realistic user scenarios for non-developers.
-
- For each:
- goal

- steps
- expected result
- pass/fail criteria.
-
- No technical jargon.

PM-15 — Code Review

This is one of the few later-stage prompts that deals with code.

- Follow CLAUDE.md.
-
- Review the current implementation of [FEATURE].
-
- Focus only on:
- correctness
- security
- authorization
- performance
- maintainability
- test coverage
-
- Do not rewrite code.
-
- Return:
- CRITICAL
- HIGH
- MEDIUM
- LOW
- APPROVED ITEMS

PM-16 — Regression Review

- Follow CLAUDE.md.
-
- Review whether the latest change can break existing OTW features.
-
- Check:
- auth
- room access
- Realtime
- cache
- members
- itinerary
- checklist
- expenses
- mobile behavior
-
- Return regression risks and required tests.

PM-17 — Pre-Release Review

- Follow CLAUDE.md.
-
- Perform OTW pre-release readiness review.
-
- Check:
- requirements

- MVP scope
- security
- performance
- mobile
- testing
- deployment
- environment variables
- documentation
- known issues
-
- Return:
- READY
- NOT READY
- BLOCKERS
- NON-BLOCKING ISSUES
-
- Do not code.

---

37. WEEKLY PROJECT MANAGEMENT ROUTINE

Start of Week

Review:

- What is the sprint goal?
- What are the must-finish tasks?
- What dependencies exist?

- What could block us?

Claude Prompt:

- Follow CLAUDE.md.
-
- Based on current project status, create the week's execution plan.
-
- Limit to highest-priority tasks required for the current milestone.
-
- Include dependencies, risks, and daily deliverables.
-
- Do not add scope.

Mid-Sprint

Run:

- Progress Review
- Risk Review
- Blocker Review

Claude Prompt:

- Follow CLAUDE.md.
-
- Review current OTW progress against the sprint goal.
-
- Identify:

- done
- behind
- blocked
- at risk
- scope creep
- next action
-
- Do not suggest new features.

End of Sprint

Do:

- Demo
- QA
- Acceptance
- Retrospective

Claude Prompt:

- Follow CLAUDE.md.
-
- Conduct a sprint retrospective for OTW.
-
- Analyze:
- what was completed
- what slipped
- root causes
- technical risks
- process problems

- what should change next sprint
-
- Keep recommendations actionable.

---

38. TEST MANAGEMENT

Create test levels:

- LEVEL 1
- Unit Testing
-
- LEVEL 2
- Integration Testing
-
- LEVEL 3
- API Testing
-
- LEVEL 4
- Frontend Testing
-
- LEVEL 5
- End-to-End Testing
-
- LEVEL 6
- Security Testing
-
- LEVEL 7
- Load Testing
-
- LEVEL 8

- UAT

Do not rely only on:

> “gumagana naman sa laptop ko.”

---

39. UAT USER SCENARIOS

A real tester should be able to perform:

Scenario 1

Create a gala.

Scenario 2

Friend joins via code.

Scenario 3

Friend changes RSVP.

Scenario 4

Host adds itinerary.

Scenario 5

Friend adds checklist item.

Scenario 6

Add expense.

Scenario 7

Verify debt settlement.

Scenario 8

Use OTW from phone.

Scenario 9

Leave and return to room.

Scenario 10

Try accessing another room.

That final scenario is extremely important.

---

40. PERFORMANCE MANAGEMENT

Do not ask:

> “Kaya ba maraming users?”

Ask measurable questions.

Test:

- 10 users
- 25 users
- 50 users
- 100 users
- 200 users

Measure:

- response time
- p95
- p99
- errors
- requests/sec
- database connections
- cache hit rate
- Redis commands
- CPU
- RAM

Also test **burst behavior**.

Example:

- 50 users suddenly join the same room

rather than only:

- 50 users slowly browse the website.

---

41. CONCURRENCY TESTING

OTW is collaborative, so race conditions matter.

Examples:

Scenario

Two users complete the same checklist item at exactly the same time.

Expected:

- one valid final state
- no corrupted data
- no duplicated record

Scenario

Two users modify RSVP simultaneously.

Expected:

- final database state is valid
- clients eventually converge

Scenario

Two users add expenses simultaneously.

Expected:

- both expenses remain
- totals update correctly

---

42. CACHE FAILURE TEST

This should be explicitly tested.

Scenario:

- Redis OFF

The application should ideally continue serving database-backed data rather than completely collapsing.

Then:

- Redis returns
- ↓
- cache resumes

This is what makes caching an optimization rather than a single point of failure.

---

43. MOBILE QA

OTW should be tested primarily on phones.

Minimum target:

- Android Chrome
- iPhone Safari
- Desktop Chrome
- Desktop Edge

Test:

- 375px
- 390px
- 412px
- 768px
- 1024px
- 1440px

Check:

- buttons
- modal
- keyboard
- forms
- map
- GPS
- photo upload
- scrolling
- text wrapping
- touch targets
- loading states

---

44. RELEASE CHECKLIST

Before production:

- [ ] MVP scope frozen
- [ ] UAT passed
- [ ] Critical bugs = 0
- [ ] High security issues = 0
- [ ] Environment variables verified
- [ ] Secrets removed from repository

- [ ] Production database configured
- [ ] Redis configured
- [ ] Auth configured
- [ ] Realtime configured
- [ ] CORS restricted
- [ ] Rate limits enabled
- [ ] File limits enabled
- [ ] Error handling verified
- [ ] Backup/recovery strategy documented
- [ ] Mobile test passed
- [ ] Load test completed
- [ ] Production smoke test completed
- [ ] README updated

---

45. DOCUMENTATION EXPECTATION

At project completion, your team should be able to show:

1. Project Charter
2. Problem Statement
3. Objectives
4. Scope
5. User Stories
6. User Flows
7. Wireframes
8. UI/Brand Guide
9. Architecture Diagram
10. ERD

- 11. API Specification
- 12. Authorization Matrix
- 13. Cache Strategy
- 14. Security Plan
- 15. Test Plan
- 16. UAT Results
- 17. Performance Results
- 18. Risk Register
- 19. Deployment Plan
- 20. User Manual

This makes the project feel like a proper software-engineering project rather than:

> “Pinagawa namin sa Claude tapos deploy.”

---

46. RECOMMENDED OTW MASTER TIMELINE

WEEK 1 — PLAN

- Day 1
- Project Charter
- Scope
-
- Day 2
- User stories
- Requirements
-
- Day 3

- User flows
- Edge cases
-
- Day 4
- Brand discovery
- UX principles
-
- Day 5
- Wireframes
- Architecture

WEEK 2 — DESIGN + FOUNDATION

- Day 6
- Database design
-
- Day 7
- API contract
-
- Day 8
- Authorization/security design
-
- Day 9
- Caching/realtime design
-
- Day 10
- Development foundation

WEEK 3 — MVP

- Day 11
- Authentication
-
- Day 12
- Create/Join Gala
-
- Day 13
- Dashboard + Members
-
- Day 14
- RSVP + Realtime
-
- Day 15
- Itinerary

WEEK 4 — COLLABORATION + MONEY

- Day 16
- Checklist
-
- Day 17
- Budget
-
- Day 18
- Expenses
-
- Day 19
- Settlement
-
- Day 20
- MVP QA

WEEK 5 — ADVANCED + HARDENING

- Day 21
- Redis/cache
-
- Day 22
- Map/GPS
-
- Day 23
- Google Drive/photos
-
- Day 24
- Fun features
-
- Day 25
- Security + performance testing

WEEK 6 — RELEASE

- Day 26
- Bug fixing
-
- Day 27
- Mobile QA
-
- Day 28
- Load testing
-
- Day 29
- UAT
-
- Day 30

- Deployment + documentation + demo

For a shorter academic deadline, remove or defer **GPS + photo vault + couple mode**, rather than sacrificing authentication, core collaboration, testing, security, and performance.

---

47. THE MOST IMPORTANT PROJECT RULE

Your team should follow this lifecycle for **every feature**:

- IDEA
- ↓
- REQUIREMENT
- ↓
- USER STORY
- ↓
- ACCEPTANCE CRITERIA
- ↓
- UX FLOW
- ↓
- TECHNICAL DESIGN
- ↓
- SECURITY CHECK
- ↓
- TASK BREAKDOWN
- ↓
- IMPLEMENTATION
- ↓
- QA

- ↓
- UAT
- ↓
- DONE

And **Claude enters the process as an assistant at each stage**, rather than being given:

> “Claude, build OTW.”

---

48. HOW TO USE CLAUDE WITHOUT WASTING TOKENS

Bad:

- Here is the entire project.
- Here are all 30 features.
- Here are 15 screenshots.
- Here is the entire database.
- Now build everything.

Better:

- Follow CLAUDE.md.
-
- Task:
- Review the approved OTW RSVP requirements.
-
- Deliver:
- acceptance criteria

- edge cases
- dependencies
- QA cases
-
- Do not code.

Then separately:

- Follow CLAUDE.md.
-
- Implement the approved RSVP design.
-
- Use the existing API and architecture.
- Do not change database structure.
- Run relevant tests.

Then separately:

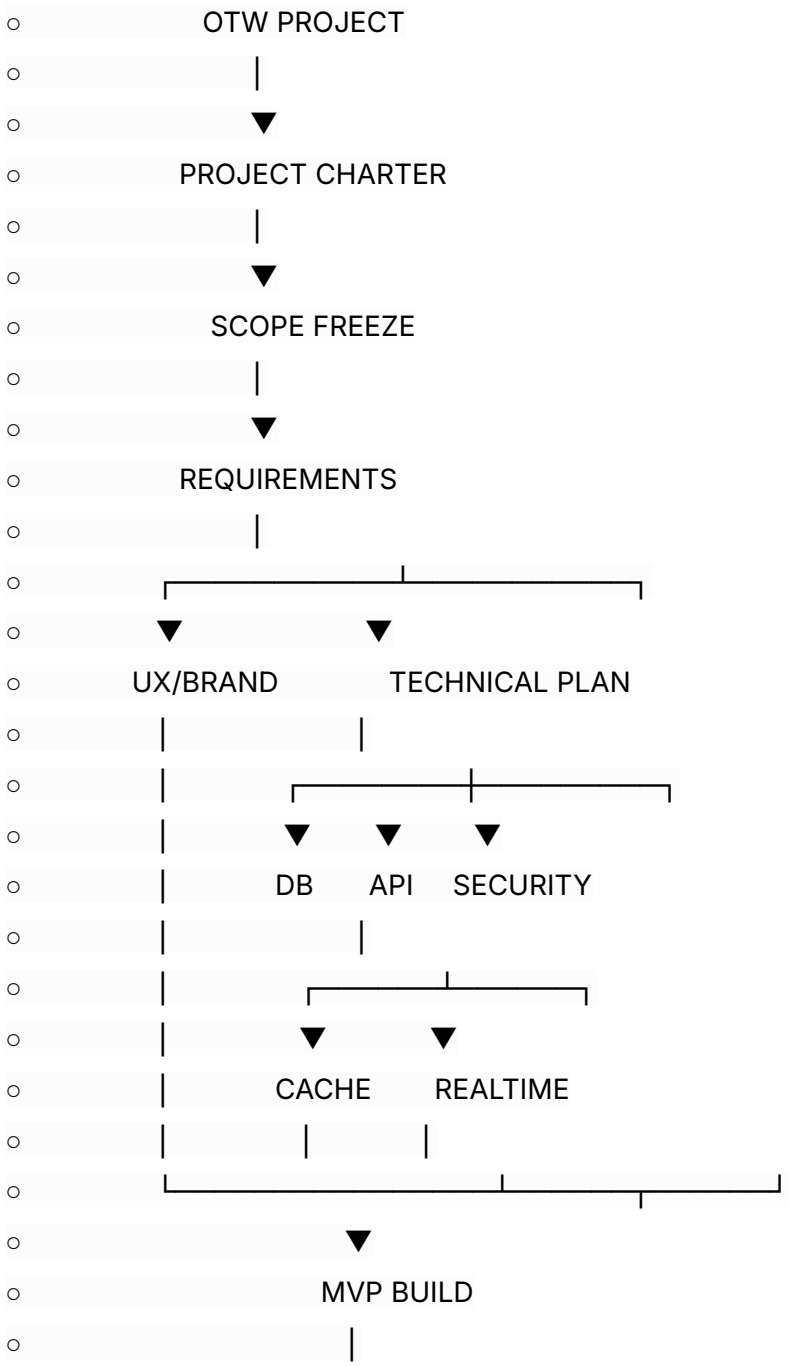
- Follow CLAUDE.md.
-
- Review the RSVP implementation.
-
- Do not modify code.
- Identify security, performance, correctness, and regression issues.

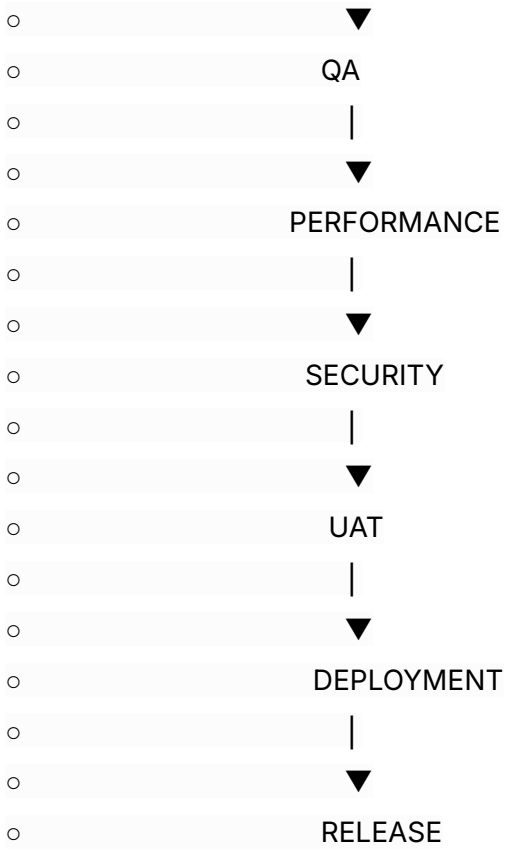
This gives Claude **small context windows with explicit objectives**, which is both easier to control and less likely to cause an unnecessary rewrite.

---

49. FINAL OTW PROJECT MANAGEMENT FLOW

The complete project now looks like this:





OTW's project priority should always be:

1. Requirements

2. Scope

3. User experience

4. Correctness

5. Security

6. Performance

7. Realtime

8. Visual polish

9. Fun extras

That order matters.

A gorgeous “OTW” that leaks another group's room data is a failed project. A functioning gala planner with clean UX, secure room isolation, reliable collaboration, proper caching, and measured performance is already a strong engineering project. Next.js is designed for interactive/dynamic React applications, and its documentation specifically discusses avoiding network waterfalls and using appropriate data-fetching patterns; that fits the performance-management approach above.

50. THE ONE RULE FOR YOUR TEAM + CLAUDE

Before Claude is allowed to build anything, you should be able to point to:

- "What requirement is this implementing?"
- "What user story is this?"
- "What is the acceptance criteria?"
- "What is the security rule?"
- "What happens when it fails?"
- "How does this affect the database?"
- "How does this affect caching?"
- "How does this affect realtime?"
- "How will we test it?"

If you cannot answer those, **the task is not ready for implementation yet.**

TEXT BRANDING

OTW — BRAND & COPY GUIDE

01. BRAND CORE

Brand Name

OTW

Official Meaning

On The Water

OTW is a catchy, memorable name built around the idea of going out, exploring, taking trips, and making plans happen.

But there's another meaning behind the name that makes the brand more playful.

In everyday conversations, **OTW** usually means:

On The Way.

And everyone knows that one friend who says:

> “OTW na ako.”

while they're still at home.

That familiar joke became part of the product itself.

OTW includes **real-time location sharing** so groups can actually see who's on the way, who's almost there, and who's... still getting ready.

So OTW has two sides:

On The Water
The brand's official meaning.

On The Way
The everyday phrase and inside joke that inspired the location feature.

---

02. BRAND PROBLEM

Friend groups are great at making plans.

The problem is actually making them happen.

“G tayo?”

“Set tayo.”

“Next weekend?”

“Wait, check ko.”

Then the conversation gets buried under hundreds of other messages in the group chat.

The plan never gets a date.

Nobody knows who's actually coming.

The itinerary is scattered across messages.

Someone ends up paying for everyone.

And on the day itself, somebody says:

> “OTW na.”

...while still at home.

OTW is designed to move the plan forward.

---

03. BRAND POSITIONING

What is OTW?

OTW is a collaborative gala planner built for friend groups who want their plans to actually happen.

It brings the important parts of a gala into one place:

- Date and schedule
- Attendance

- Itinerary
- Budget estimation
- Expense tracking
- Checklist
- Real-time location sharing
- Gala photos
- Memories
- Calendar
- Couple Mode

No complicated setup.

No account required just to start planning or join a gala.

Product Promise

> **From planning the gala to actually going.**

---

04. THE BIG BRAND IDEA

From “G tayo?” to “OTW na.”

This is the central idea behind the entire brand.

“G tayo?” represents the idea.

Planning turns the idea into something real.

“OTW na.” represents the action.

And with OTW's location sharing:

> **This time, we know you're actually OTW.**

The entire user journey should feel like moving from one stage to the next:

G tayo?

↓

When are we going?

↓

Who's G?

↓

What's the plan?

↓

How much are we spending?

↓

What do we need?

↓

Where's everyone?

↓

OTW na.

↓

You made it happen.

---

05. BRAND TAGLINE SYSTEM

Instead of using multiple taglines equally, OTW has one primary line supported by secondary messaging.

PRIMARY BRAND LINE

> **From “G tayo?” to “OTW na.”**

BRAND PROMISE

> **Plan it. Share it. Make it happen.**

CORE SUPPORTING LINE

Plans are better when they leave the group chat.

LOCATION FEATURE LINE

OTW na ba talaga?

Supporting:

Now you can actually see.

CAMPAIGN LINES

These can be used for social media, promotional graphics, banners, and marketing campaigns:

Less “set tayo.” More “OTW na.”

Your next gala shouldn't stay in the GC.

Because “G!” deserves an actual plan.

Stop saying “someday.” Start planning it.

May plano na. Let's make it happen.

No more fake OTWs.

Your GC says OTW. We say, “Where are you?”

---

06. BRAND PERSONALITY

OTW should feel like:

That one friend in the group chat who actually makes the plan happen.

Not the loudest friend.

Not the overly enthusiastic friend.

Not a corporate travel assistant.

Not a brand desperately trying to sound Gen Z.

OTW is:

Casual
Clever
Warm
Confident
Organized
Encouraging

The personality should feel like:

> “Okay, enough talking. Let's actually plan this.”

Not:

> “OMG BESTIES LET'S GOOOO!!!”

OTW can be playful without becoming childish.

---

07. LANGUAGE & COPY STYLE

The sweet spot: Taglish.

Use English for clarity and modern product language, then Filipino naturally where it adds personality.

The copy should sound like something people would actually say.

Good:

> Finally, a place for all those gala plans na ilang buwan nang nasa group chat.
>
> Saan tayo? Let's actually put it on the calendar.
>
> May date na. May budget na. Next step: actually showing up.
>
> Okay, who's actually G?

Avoid overly formal Filipino:

> Planuhin ang inyong paglalakbay kasama ang inyong mga kaibigan.

Avoid forced Gen Z:

> OMG BESTIES LET'S G NA FR FR!!!

OTW should feel **naturally conversational**, not like it's trying to prove that it's young.

---

08. LANDING PAGE

HERO

Eyebrow

FOR PLANS THAT ACTUALLY HAPPEN

Main Headline

From “G tayo?” to “OTW na.”

Supporting Copy

Finally, a place for all those gala plans na ilang buwan nang nasa group chat.

Brand Promise

Plan it. Share it. Make it happen.

CTA

Plan a Gala

Join a Gala

Optional microcopy

> **And yes, we can tell if you're actually OTW.**

This line can be used as a small supporting element rather than part of the main hero if the design needs to stay minimal.

---

09. LANDING PAGE — SECOND SECTION

Headline

Your gala starts here.

Copy

> Pick a date, get the tropa, build the itinerary, sort out the budget, and keep everything in one place.
>
> No complicated setup. Just make a plan and get everyone moving.

---

10. PLAN A GALA

When someone chooses **Plan a Gala**, the tone becomes more helpful and conversational.

STEP 1 — GALA NAME

First things first. What are we calling this gala?

Placeholder:

`e.g. Baguio Before Graduation`

Helper:

> Give your gala a name your GC will remember.

STEP 2 — DATE

When are we going?

Helper:

> Let's turn “someday” into an actual date.

STEP 3 — DESTINATION

Where are we headed?

Helper:

> Add the destination. We'll figure out the rest together.

STEP 4 — PEOPLE

Who's coming?

Helper:

> Send the gala code to the group and let them join.

---

11. GALA CODE

After creating the gala:

Header

Okay, this is happening.

Supporting Copy

> Your gala is ready. Share the code with your friends so they can jump in.

Example Code

OTW-7K2P

CTA

Copy Gala Code

Secondary:

Enter the Gala

Small Text

> No account needed. Just share the code and get the group together.

---

12. JOIN A GALA

Header

Got a code? Let's go.

Supporting Copy

> Enter the gala code from your friend and you're in.

Input:

`Enter gala code`

Button:

Join the Gala

Success

> **You're in. See you at the gala.**

---

13. PLANNER DASHBOARD

The dashboard should be more functional than promotional.

Personality stays in the microcopy.

Header

Let's get this gala together.

DATE

When are we going?

PEOPLE

Who's G?

ITINERARY

What's the plan?

BUDGET

How much are we looking at?

CHECKLIST

What are we bringing?

LOCATION

Where is everyone?

PHOTOS

What did we get?

---

14. ATTENDANCE

Instead of:

Attendance Status

Use:

So... who's G?

Options:

I'm G

Maybe

Can't make it

I'm G

Nice. You're officially on the list.

Maybe

We'll keep you in the maybe pile.

Can't make it

Sayang. Next gala na lang.

---

15. ITINERARY

Header

Okay, what's the plan?

Supporting Copy

Add the places, activities, food stops, and everything in between.

CTA

Add a Stop

Empty State

No stops yet. Saan tayo?

Itinerary Name

The Gala Route

---

16. CHECKLIST

Header

Don't forget the important stuff.

Supporting Copy

Because someone always forgets the charger.

CTA

Add to Checklist

Empty State

Nothing here yet. Pack smart.

---

17. BUDGET ESTIMATOR

Header

Let's talk gastos.

Supporting Copy

Estimate the budget before anyone starts saying, “Magkano ba kailangan?”

Categories

Transportation

Stay

Food

Activities

Other

Result

Estimated budget per person

₱2,450

Supporting:

> **Looks doable. Ready to lock it in?**

---

18. EXPENSE TRACKER

Header

Okay, sino nag-abono?

Expense Information

Total spent

Your share

You've paid

Remaining

When everything is settled

> **All settled. Walang may habol.**

---

19. REAL-TIME LOCATION

This feature is especially important to OTW's identity.

Feature Name

Who's actually OTW?

Supporting Copy

> Less “Nasaan ka na?” More “2 minutes away.”

CTA

Share My Location

Active Status

> **You're sharing your location.**

Other statuses

Still at home

On the way

Almost there

Already there

The actual location, privacy, and permission information should remain clear and straightforward.

The humor should support the feature, never hide what the feature does.

---

20. LOCATION CAMPAIGN

This feature can also become a recurring OTW marketing concept.

Campaign

“OTW na raw.”

Supporting:

> We've all heard it.

Then:

> **This time, there's a map.**

Another version:

OTW na ba talaga?

> Now you can actually see.

Another:

No more fake OTWs.

> Share your location and let the map do the talking.

---

21. GALA DUMP

Instead of simply calling it:

Photos

OTW calls it:

The Gala Dump

Supporting Copy

All the good shots, bad shots, random candids, and “send mo nga” moments in one place.

CTA

Add Photos

Empty State

No memories yet. Go make some.

After the Gala

Another gala for the books.

---

22. COUPLE MODE

Header

Just the two of you?

Supporting Copy

Same OTW energy. Smaller passenger count.

Features

Date Ideas

Places to Go

Budget for Two

Our Photos

---

23. CALENDAR

Calendar becomes one of the account-based features.

Header

Your Gala Calendar

Supporting Copy

> Keep track of what's coming up, what's being planned, and what's already happened.

Empty State

> **Nothing on the calendar yet.**

Supporting:

> Sounds like a good time to plan something.

---

24. ACCOUNT / SIGN-UP

OTW should not force an account just to use the basic planner.

Product Philosophy

> **You can start planning without us asking for your life story.**

When introducing accounts:

Want to keep your galas with you?

Supporting:

> Create an account to save your galas, keep a personal calendar, view your gala history, and get reminders.

CTA:

Create an Account

Secondary:

Maybe Later

Small reassurance:

> **Your current gala can still continue without an account.**

---

25. WHAT WORKS WITHOUT AN ACCOUNT

Users should be able to start with minimal friction.

Available without an account

- Create a gala
- Join a gala
- View the gala
- Set attendance
- Add/edit itinerary
- Use checklist
- Use budget estimator
- Track expenses
- Share location
- View and potentially upload gala photos, depending on the storage implementation

The exact persistence of these features depends on how the gala session and Google Drive integration are implemented.

---

26. WHAT AN ACCOUNT UNLOCKS

The account should exist primarily for **personal persistence**, not basic access.

Account features

Personal Gala Calendar

Keep all your plans in one place.

Saved Galas

Come back to your plans anytime.

Gala History

See the places you've been.

Personal Profile

Your OTW space.

Website Notifications

Get reminders even when you're not inside OTW.

This creates a natural reason to sign up without making registration a barrier.

---

27. NOTIFICATION COPY

For users who choose to enable notifications:

Gala Coming Up

Gala coming up

Your Baguio trip is 3 days away.

Someone Joined

Someone joined

Your gala just got one person bigger.

Checklist

Quick check

May nakalimutan ka ba?

Budget

Budget update

Your estimated gala budget has changed.

Day Before

Tomorrow's the day.

Charge your phone. Pack your things. Don't be that friend.

Gala Day

Today's the day.

Time to go.

---

28. COMPLETED GALA

After the gala:

Header

You made it happen.

Supporting Copy

From “G tayo?” to an actual day on the calendar.

Actions

View Gala Dump

View Expenses

View Itinerary

Plan Another Gala

Closing Line

> **Until the next “G tayo?”**

This creates a natural loop back to the beginning of the OTW experience.

---

29. EMPTY STATE COPY KIT

Situation	OTW Copy
No galas	**Wala pang ganap.**
No itinerary	**So... saan tayo?**
No members	**Ikaw pa lang ang G.**
No photos	**No memories yet. Go make some.**
No expenses	**Wala pang gastos. Enjoy habang libre pa.**
No checklist	**Nothing here yet. Pack smart.**
No upcoming plans	**Looks like the calendar needs a little excitement.**
Gala completed	**You made it happen.**
All expenses settled	**All settled. Walang may habol.**
Someone joins	**You're in. See you at the gala.**
Invalid code	**Hmm, hindi namin makita 'yan. Check the code and try again.**

---

30. COPY PRINCIPLES

OTW should not constantly tell people:

> “Have fun!”

The copy should instead **move the user toward action.**

The progression should feel intentional:

IDEA

> “G tayo?”

COMMITMENT

> “When are we going?”

PLANNING

> “Okay, what's the plan?”

COORDINATION

> “Who's G?”

PREPARATION

> “What do we need?”

EXECUTION

> “Where's everyone?”

MEMORY

> “The Gala Dump”

COMPLETION

> “You made it happen.”

---

31. OTW'S COPY RULE

Every line doesn't need to be funny.

That's important.

If every button, heading, notification, and label tries to make a joke, the product starts feeling gimmicky.

Instead:

70% clear and conversational

20% playful

10% memorable / witty

The personality should come through naturally.

The product should always remain easy to understand.

---

32. BRAND VOICE IN ONE SENTENCE

> **OTW sounds like the organized friend who says “G” and actually makes the plan.**

---

33. THE OTW BRAND LOOP

The strongest part of the concept is that the name and product naturally connect.

Someone says:

“G tayo?”

OTW helps them:

Plan it.

Then:

Share it.

Then:

Get everyone moving.

Then:

See who's actually OTW.

Then:

Go.

Then:

Save the memories.

And eventually:

Plan the next one.

That is the OTW loop.

---

34. FINAL BRAND MESSAGE

OTW

On The Water

A collaborative gala planner for plans that deserve to leave the group chat.

From “G tayo?” to “OTW na.”

Plan the date.
Get the tropa.
Build the itinerary.
Sort out the budget.
Know who's coming.
See who's actually on the way.
Keep the memories.

No complicated setup.

No need to create an account just to start.

Just make a plan and get everyone moving.

Brand Promise

Plan it. Share it. Make it happen.

Core Thought

Plans are better when they leave the group chat.

Brand Personality

Fun. Light. Smart. Warm. Encouraging.

Brand Voice

The organized friend in your group chat.

Signature Line

From “G tayo?” to “OTW na.”

The OTW Twist

On The Water is the official meaning.

On The Way is the familiar phrase.

“OTW na.” is the joke.

Real-time location sharing is the proof.

And the ultimate goal?

You actually went.

PROJECT TASKING

WEEK 1 — Foundation + No-Login Core Flow

D1-01 — Repo, branching, and environment scaffolding Role: [DB/DevOps]

- **Objective:** Get the team a shared, working repo before anyone writes feature code.
- **Flow:** Developer clones repo → runs `npm install` → duplicates `.env.example` to `.env` → inputs local/dev Supabase keys → runs `npm run dev` → sees running localhost shell.
- **Tasks:**
  - Create repo with main/dev/feature/* branch convention
  - Create Supabase project (dev), capture keys into `.env.example` (no real secrets committed)
  - Add root README.md stub and docs/ folder per Master Plan §9
- **Acceptance Criteria:**
  - Any teammate can clone, copy `.env.example` → `.env`, and run the app locally
  - No secrets present in git history
- **Dependencies:** None

D1-02 — Spring Boot service scaffold Role: [BE]

- **Objective:** Stand up the backend skeleton so endpoints have somewhere to live.
- **Flow:** API Request arrives → Global Exception Handler intercepts errors OR Controller processes success → Returns standard `{ data, error, meta }` JSON envelope.
- **Tasks:**
  - Scaffold packages: room, member, auth, common
  - Define a standard API response envelope (`{ data, error, meta }`) used by every endpoint from Day 1 onward
- **Acceptance Criteria:**
  - Health-check endpoint returns the standard envelope
  - Package structure matches this ticket, not ad hoc per-dev naming
- **Dependencies:** D1-01

D1-03 — Next.js frontend scaffold Role: [FE]

- **Objective:** Stand up the frontend skeleton with the real stack, not a generic starter.
- **Flow:** User navigates URL → Next.js App Router maps `/` (Landing), `/plan` (Plan-a-Gala), `/join` (Join), `/[code]` (Dashboard) → Returns blank page skeletons.
- **Tasks:**
  - Init project with Next.js + TypeScript + Shadcn + TanStack Query
  - Create route placeholders: Landing, Plan-a-Gala flow, Join-a-Gala flow, Leader Dashboard, Member Dashboard
- **Acceptance Criteria:**
  - App builds and runs locally with all placeholder routes reachable

- **Dependencies:** D1-01

D1-04 — rooms table design & migration Role: [DB]

- **Objective:** Persist gala rooms.
- **Flow:** BE Service calls DB → Migration script executes → `rooms` table is created with unique constraints active.
- **Tasks:**
  - Create columns: id, title, gala_code, status, leader_anon_id, created_at
  - Set Unique constraint on gala_code
- **Acceptance Criteria:**
  - Migration runs cleanly
  - gala_code collisions are rejected at the DB level
- **Dependencies:** D1-01

D1-05 — room_members table design & migration Role: [DB]

- **Objective:** Persist who is in which room.
- **Flow:** Member joins room → DB inserts row in `room_members` → Room is deleted → Cascade delete removes associated `room_members` rows automatically.
- **Tasks:**
  - Create columns: id, room_id, anon_user_id, nickname, role (leader/member), joined_at
  - Set FK to rooms, cascade delete on room removal
- **Acceptance Criteria:**
  - A room can have many members
  - Deleting a room cleans up its members automatically
- **Dependencies:** D1-04

D1-06 — Brand-driven UI theme tokens (Bubblegum Chaos) Role: [FE]

- **Objective:** Bake the brand personality (casual, warm, organized) into the design tokens before any screen gets built.
- **Flow / UI Integration:** Shadcn components render → Inherit CSS variables (`--rosy`, `--zest`, `--teal`, `--blossom`, `--cream`) → Apply `Fredoka` font for headings and `Nunito` for body → Buttons apply 999px rounded borders and box-shadow depth.
- **Tasks:**
  - Map exact hex codes from the HTML UI file to `tailwind.config.ts`
  - Set up next/font for Fredoka and Nunito
  - Configure organic blob border-radiuses as custom Tailwind utilities
  - Wire tokens into the Shadcn theme provider
- **Acceptance Criteria:**
  - A test page rendering all tokens looks consistent and matches the intended tone (strictly avoiding default Shadcn slate/gray)

- **Dependencies:** D1-03

D1-07 — GitHub Project board + issue templates Role: [QA/PM]

- **Objective:** The board itself needs to exist before the backlog is usable.
- **Flow:** PM clicks "New Issue" → Selects OTW Template → Fills Objective/Scope/Flow/Tasks → Saves to BACKLOG column.
- **Tasks:**
  - Create the 3 GitHub Projects (Week 1/2/3) with BACKLOG → READY → IN PROGRESS → REVIEW → QA → UAT → DONE (+ BLOCKED) columns
  - Save an issue template with this doc's fields
- **Acceptance Criteria:**
  - New issues can be created from the template in under a minute
- **Dependencies:** None

D1-08 — CI: build + lint on PR Role: [DevOps]

- **Objective:** Catch broken builds before merge, from day one.
- **Flow:** Dev opens PR to `dev` branch → GitHub Actions triggers → Runs `npm run lint` & `npm run build` (FE) + `mvn test` (BE) → Blocks merge if red.
- **Tasks:**
  - FE: install, lint, build on every PR
  - BE: compile + unit test run on every PR
- **Acceptance Criteria:**
  - A PR with a broken build is blocked from merging to dev
- **Dependencies:** D1-02, D1-03

D2-01 — Silent anonymous session on entry Role: [BE]

- **Objective:** Give every visitor a stable identity without ever showing a login screen.
- **Flow:** User opens app → Middleware checks for session token → If none, triggers Supabase anonymous auth → Stores `anon_user_id` in secure cookie/local storage → Uses ID for all subsequent room creations/joins.
- **Tasks:**
  - Wire Supabase anonymous auth into the backend's session handling
  - Confirm the resulting anon_user_id is stable across requests in the same browser session
- **Acceptance Criteria:**
  - No UI element ever asks for an email/password to use core features
  - Re-opening the app in the same browser reuses the same anon identity
- **Dependencies:** D1-02

D2-02 — Gala code generator Role: [BE]

- **Objective:** Produce the shareable code that replaces login as the access mechanism.

- **Flow:** Room creation triggered → Generator creates `OTW-XXXX` string → Checks DB for collision → If collision, regenerates (max 3 retries) → Returns unique code to controller.
- **Tasks:**
    - Generate 6-char alphanumeric codes in the `OTW-XXXX` format
    - Retry on collision against the gala_code unique constraint
- **Acceptance Criteria:**
    - 10,000 generated codes in a local test produce zero unhandled collisions
- **Dependencies:** D1-04

D2-03 — RLS policies for rooms & members Role: [DB]

- **Objective:** Enforce room isolation at the database level.
- **Flow:** API queries rooms table → Supabase evaluates JWT anon_user_id → Checks if ID exists in room_members for that room → Returns data if true, returns empty/403 if false.
- **Tasks:**
    - Policy: a session may only read/write member rows for rooms it belongs to
    - Policy: a session may only read room rows it's a member of
- **Acceptance Criteria:**
    - A direct Supabase query using another session's token cannot read a room it isn't part of
- **Dependencies:** D1-04, D1-05, D2-01

D2-04 — POST /api/v1/rooms (create) Role: [BE]

- **Objective:** Turn "Plan a Gala" into a real room.
- **Flow:** FE sends title + nickname → BE validates input → Calls D2-02 for Code → Inserts to rooms → Inserts creator to room_members as leader → Returns `{ room, code }`.
- **Tasks:**
    - Accept title, create room + gala code, auto-add creator as leader
    - Return the created room + code in the standard envelope
- **Acceptance Criteria:**
    - Calling this endpoint twice with the same title never creates duplicate leader memberships
- **Dependencies:** D2-01, D2-02, D2-03

D2-05 — Landing page (UI Integration) Role: [FE]

- **Objective:** Ship the first thing anyone sees, in the brand's actual voice.
- **Flow / Brand Integration:** User navigates to `/` → Views Bubblegum Chaos Hero Section → Clicks "Plan a Gala" button → Transitions smoothly to `/plan`.
- **Tasks:**
    - Implement the hero section per Brand Guide §08

    - Headline: "From 'G tayo?' to 'OTW na.'"
    - Supporting copy: "Finally, a place for all those gala plans na ilang buwan nang nasa group chat."
    - CTAs: Plan a Gala / Join a Gala
- **Acceptance Criteria:**
    - Matches §08 copy exactly
    - Both CTAs route to their respective flows
- **Dependencies:** D1-03, D1-06

D2-06 — "Plan a Gala" Step 1: Gala Name Role: [FE]

- **Objective:** First step of gala creation, in-voice.
- **Flow / Brand Integration:** User lands on `/plan` → Sees "First things first. What are we calling this gala?" → Types name (e.g. "Baguio Before Graduation") → Clicks "Next" → FE calls `POST /rooms` (D2-04) → Routes to `/code-confirm`.
- **Tasks:**
    - Header: "First things first. What are we calling this gala?"
    - Placeholder: e.g. `Baguio Before Graduation`; helper: "Give your gala a name your GC will remember."
    - Wire to D2-04 creation API
- **Acceptance Criteria:**
    - Submitting calls D2-04 and routes to the Gala Code screen (D3-02) on success
- **Dependencies:** D2-04

D2-07 — QA: code collision & bad-title tests Role: [QA]

- **Objective:** Catch the failure modes before they reach a demo.
- **Flow:** Automated test sends 100 char title → Expects 400 Bad Request envelope. Test mocks DB to force `gala_code` collision → Expects successful retry and 201 Created.
- **Tasks:**
    - Test forced code collision retry
    - Test empty/overlong title rejection
- **Acceptance Criteria:**
    - Both cases fail gracefully with a clear error in the envelope, never a raw 500
- **Dependencies:** D2-02, D2-04

D2-08 — Decide & document: gala code expiry rule Role: [PM]

- **Objective:** Close one of the open questions from the MVP plan.
- **Flow:** Team syncs → Decides expiry logic (e.g., +3 days post-gala date) → Updates `CHANGE_LOG.md` → BE/DB devs apply constraints in future sprint.
- **Tasks:**
    - Team decides: auto-expire N days after gala date, or manual close by leader
    - Record the decision in `CHANGE_LOG.md`

- **Acceptance Criteria:**
  - Decision is written down with a reason, not left implicit
- **Dependencies:** None

D3-01 — POST /api/v1/rooms/join Role: [BE]

- **Objective:** Let a code turn into membership.
- **Flow:** FE sends `code` + `nickname` → BE checks if `code` exists/active → If valid, inserts user into `room_members` with role `member` → Returns room data.
- **Tasks:**
  - Validate code exists, isn't expired, room isn't full
  - Add member with submitted nickname, role `member`
- **Acceptance Criteria:**
  - Invalid code returns a specific "not found" error, not a generic failure
- **Dependencies:** D2-02, D2-03

D3-02 — Gala Code confirmation screen Role: [FE]

- **Objective:** Give the leader something shareable, in-voice, immediately after creation.
- **Flow / Brand Integration:** Renders `/code-confirm` → Displays "Okay, this is happening." and code → User clicks "Copy Gala Code" → Clipboard API copies code → User clicks "Enter the Gala" → Routes to Dashboard.
- **Tasks:**
  - Header: "Okay, this is happening." / copy: "Your gala is ready. Share the code with your friends so they can jump in."
  - Show code (e.g. `OTW-7K2P`) with **Copy Gala Code** button; secondary **Enter the Gala**
  - Small text: "No account needed. Just share the code and get the group together."
- **Acceptance Criteria:**
  - Copy button copies the exact code to clipboard
  - "Enter the Gala" routes to Leader Dashboard
- **Dependencies:** D2-06

D3-03 — Join a Gala screen Role: [FE]

- **Objective:** Mirror the creation flow's simplicity for joiners.
- **Flow / Brand Integration:** User clicks "Join a Gala" → Routes to `/join` → Reads "Got a code? Let's go." → Inputs code + nickname → Submits → Sees "You're in. See you at the gala." → Routes to Dashboard.
- **Tasks:**
  - Header: "Got a code? Let's go."; input placeholder `Enter gala code`; button **Join the Gala**
  - Success state: "You're in. See you at the gala."
  - Wire to D3-01

- **Acceptance Criteria:**
  - Valid code → success copy shown, routes to nickname step then Member Dashboard
- **Dependencies:** D3-01

D3-04 — Nickname-per-room uniqueness Role: [BE]

- **Objective:** Avoid two "Miguel"s causing confusion in the same room.
- **Flow:** `POST /join` executed → DB checks room_members for exact nickname match in that room_id → If match, appends number (e.g., "Miguel (2)") or rejects with 409 prompting user to change it.
- **Tasks:**
  - Reject duplicate nicknames within the same room
  - Suggest a disambiguated variant
- **Acceptance Criteria:**
  - Two joins with the same nickname in one room never both succeed silently
- **Dependencies:** D3-01

D3-05 — Empty-state & error copy pass Role: [FE]

- **Objective:** Every dead end in the flow should sound like OTW, not like a stack trace.
- **Flow / Brand Integration:** API returns 404 for code → FE intercepts error → Maps to Brand Guide §29 ("Hmm, hindi namin makita 'yan...") → Renders in standard UI toast/alert.
- **Tasks:**
  - Invalid code: "Hmm, hindi namin makita 'yan. Check the code and try again."
  - Wire this into the Join screen's error state
- **Acceptance Criteria:**
  - No raw API error text is ever shown directly to the user
- **Dependencies:** D3-03

D3-06 — QA: cross-session create + join Role: [QA]

- **Objective:** Validate the actual Week 1 exit criteria early.
- **Flow:** Window A (Incognito 1) creates room → Gets Code. Window B (Incognito 2) navigates to `/join` → Inputs Code → Both windows view the exact same Dashboard state.
- **Tasks:**
  - Two incognito windows: one creates, one joins by code
  - Confirm both see the same room id/members
- **Acceptance Criteria:**
  - Test passes with zero login screen encountered in either window
- **Dependencies:** D2-04, D3-01

D3-07 — Unique index on gala_code Role: [DB]

- **Objective:** Belt-and-suspenders against the code generator's edge cases.
- **Flow:** Concurrent requests generate identical codes → First commits → Second hits DB unique index constraint → DB throws error → BE generator catches and retries.
- **Tasks:**
    - Confirm unique index exists and is enforced under concurrent inserts
- **Acceptance Criteria:**
    - Concurrent create requests with a forced same-code scenario: only one succeeds
- **Dependencies:** D1-04

D4-01 — Leader dashboard shell Role: [FE]

- **Objective:** Give the leader a home base, even before features have data.
- **Flow / Brand Integration:** User enters dashboard → FE checks role == `leader` → Renders Fredoka H1 "Let's get this gala together." → Renders 7 empty state blobs (Date, People, Itinerary, etc.) using Bubblegum UI styling.
- **Tasks:**
    - Header: "Let's get this gala together."
    - Sections with labels: Date ("When are we going?"), People ("Who's G?"), Itinerary ("What's the plan?"), Budget ("How much are we looking at?"), Checklist ("What are we bringing?"), Location ("Where is everyone?"), Photos ("What did we get?")
- **Acceptance Criteria:**
    - All 7 sections render with correct headers and an empty-state placeholder each
- **Dependencies:** D3-02

D4-02 — Member dashboard shell Role: [FE]

- **Objective:** Same info, member permissions.
- **Flow:** User enters dashboard → FE checks role == `member` → Renders D4-01 components but hides leader-only edit buttons/toggles.
- **Tasks:**
    - Reuse D4-01, conditionally hide admin controls
- **Acceptance Criteria:**
    - Visually consistent with leader board, minus admin affordances
- **Dependencies:** D4-01

D4-03 — Role-based authorization middleware Role: [BE]

- **Objective:** Leader vs. Member backend guarantee.
- **Flow:** Request hits protected endpoint (e.g., edit room) → Interceptor extracts anon_user_id + room_id → Queries room_members role → If `member`, throws 403 Forbidden → Else proceeds to controller.
- **Tasks:**
    - Reusable check for leader role

    - Apply to stub endpoint
- **Acceptance Criteria:**
    - Direct API call from member to leader-stub rejected
- **Dependencies:** D2-03

D4-04 — GET /api/v1/rooms/:code Role: [BE]

- **Objective:** Give the dashboard real data.
- **Flow:** FE mounts `/dashboard/[code]` → Calls `GET /rooms/:code` → BE verifies session is in `room_members` → Returns Room JSON + Members Array.
- **Tasks:**
    - Return room details + member list for valid session
- **Acceptance Criteria:**
    - Unauthorized session gets 403, not room data
- **Dependencies:** D4-03

D4-05 — Wire dashboards to live data Role: [FE]

- **Objective:** Replace shell placeholders with the real room/member state.
- **Flow:** TanStack Query `useQuery` fetches room data → Hydrates Dashboard Shells → React state maps DB status to appropriate UI text.
- **Tasks:**
    - Fetch room + members on load
    - Render data into D4-01/D4-02 shells
- **Acceptance Criteria:**
    - Refreshing the dashboard shows correct, current data every time
- **Dependencies:** D4-01, D4-02, D4-04

D4-06 — QA: role-gating verification Role: [QA]

- **Objective:** Confirm backend security holds.
- **Flow:** Extract Member session token → Open Postman → Send `POST` to Leader-only stub endpoint with Member token → Assert 403 status code is returned.
- **Tasks:**
    - API request via Postman/cURL bypassing UI
- **Acceptance Criteria:**
    - Rejected every time, logged as passing security test
- **Dependencies:** D4-03

D4-07 — Staging deploy Role: [DevOps]

- **Objective:** Demoable daily progress.
- **Flow:** Code pushed to `dev` → Vercel builds FE → Render builds Spring Boot BE → Both connect to Supabase Dev DB → Team receives staging URL.
- **Tasks:**

    - Vercel preview (FE), Render deploy (BE), Supabase dev
- **Acceptance Criteria:**
    - Teammate can open staging URL on phone and complete flow
- **Dependencies:** D1-08

D5-01 — member.joined realtime event Role: [BE]

- **Objective:** Room reacts when someone joins.
- **Flow:** Join endpoint completes DB insert → BE triggers Supabase Realtime publish → Channel `room:[code]` receives `member.joined` payload containing new member data.
- **Tasks:**
    - Publish `member.joined` to private Supabase channel
- **Acceptance Criteria:**
    - Fires exactly once per successful join
- **Dependencies:** D3-01

D5-02 — Live member list Role: [FE]

- **Objective:** Prove realtime works end-to-end before building on top of it.
- **Flow:** Dashboard mounts → FE subscribes to Supabase channel `room:[code]` → Listens for `member.joined` → Appends payload to TanStack Query cache → UI re-renders "Who's G?" list automatically.
- **Tasks:**
    - Subscribe to the room channel
    - Append new members to the list without a refresh
- **Acceptance Criteria:**
    - A second device joining appears on the first device's screen within a couple seconds, unrefreshed
- **Dependencies:** D5-01, D4-05

D5-03 — Rate limiting on create/join Role: [BE]

- **Objective:** Resist code brute-forcing.
- **Flow:** Request hits `/join` → Rate limiter checks IP/Session hash → If > 10 requests/min, returns 429 Too Many Requests → FE displays "Slow down."
- **Tasks:**
    - IP/session-based rate limit on create/join
- **Acceptance Criteria:**
    - Rapid repeated joins against wrong code get throttled
- **Dependencies:** D2-04, D3-01

D5-04 — Security check: unauthorized room access Role: [QA]

- **Objective:** Master Plan UAT Scenario 10.

- **Flow:** Get token for User A in Room 1 → Attempt `GET /rooms/CODE2` (Room 2) → Verify both BE Controller and Supabase RLS block the request.
- **Tasks:**
  - Attempt to load room data using a session that never joined it
- **Acceptance Criteria:**
  - Fails at API and RLS layers
- **Dependencies:** D2-03, D4-03

D5-05 — Mobile pass on core flow Role: [QA]

- **Objective:** Phones-first validation.
- **Flow:** Open Chrome DevTools → Set viewport to 375px (iPhone SE) → Navigate Landing -> Plan -> Dashboard → Verify Fredoka H1s don't break lines awkwardly and blobs don't cause horizontal scroll.
- **Tasks:**
  - Test at 375px, 390px, 768px
- **Acceptance Criteria:**
  - No overflow, unreadable text, or unreachable buttons
- **Dependencies:** D2-05, D3-02, D3-03, D4-01

D5-06 — Week 1 exit-criteria demo + retro Role: [PM]

- **Objective:** Confirm week delivered promises.
- **Flow:** Team syncs → PM shares screen → Opens two incognito windows → Runs create/join flow live without a single login prompt → Notes carryover tasks for Week 2.
- **Tasks:**
  - Live demo; log carryover
- **Acceptance Criteria:**
  - Week 1 exit criteria demonstrably pass
- **Dependencies:** All Week 1 tickets

D5-07 — Tag week-1-complete Role: [DevOps]

- **Objective:** Mark milestone in git.
- **Flow:** Merge final Day 5 PRs to `dev` → Run `git tag -a v1.0-week1 -m "Week 1 Complete"` → `git push origin --tags`.
- **Tasks:**
  - Tag dev at Day 5 state
- **Acceptance Criteria:**
  - Tag exists and is referenced in retro notes
- **Dependencies:** D5-06

WEEK 2 — Planner Core (Collaboration + Money)

(Core logic and UI/Brand integration for the remaining planner features)

D6-04 — "So... who's G?" screen Role: [FE]

- **Objective:** Ship the attendance UI in the brand's actual voice, per §14.
- **Flow / Brand Integration:** User views "People" section → Sees Fredoka header "So... who's G?" → Taps "I'm G" → State updates → UI renders inline success copy "Nice. You're officially on the list." in Nunito font.
- **Tasks:**
  - Options: `I'm G` / `Maybe` / `Can't make it`
  - Response copy: "Nice. You're officially on the list." / "We'll keep you in the maybe pile." / "Sayang. Next gala na lang."
- **Acceptance Criteria:**
  - Selecting an option calls D6-02 and shows the matching response copy
- **Dependencies:** D6-02

D7-04 — Itinerary screen Role: [FE]

- **Objective:** Ship it in-voice, per §15.
- **Flow / Brand Integration:** User scrolls to Itinerary → Renders "Okay, what's the plan?" → If empty, shows Bubblegum UI card with "No stops yet. Saan tayo?" → Clicking "Add a Stop" reveals form.
- **Tasks:**
  - Header: "Okay, what's the plan?"; CTA **Add a Stop**
  - Empty state: "No stops yet. Saan tayo?"
- **Acceptance Criteria:**
  - Empty room shows the empty state; adding a stop replaces it with the list
- **Dependencies:** D7-02

D9-03 — Budget screen Role: [FE]

- **Objective:** Ship in-voice, per §17.
- **Flow / Brand Integration:** User views Budget section → Sees "Let's talk gastos." → Inputs values into Category fields → TanStack queries aggregate total → UI displays "Estimated budget per person ₱X,XXX" with sub-text "Looks doable. Ready to lock it in?"
- **Tasks:**
  - Header: "Let's talk gastos."; categories: Transportation, Stay, Food, Activities, Other
  - Result copy pattern: "Estimated budget per person" + amount, supporting "Looks doable. Ready to lock it in?"
- **Acceptance Criteria:**
  - Submitting an estimate updates the aggregate shown to the whole room
- **Dependencies:** D9-02

D10-02 — Expense summary + settled state Role: [FE]

- **Objective:** Ship in-voice, per §18.

- **Flow / Brand Integration:** Calculations return from D10-01 backend → FE renders "Okay, sino nag-abono?" → If remaining balance for all users == 0, renders vibrant UI blob with text "All settled. Walang may habol."
- **Tasks:**
  - Fields: Total spent, Your share, You've paid, Remaining
  - Fully settled state: "All settled. Walang may habol."
- **Acceptance Criteria:**
  - Numbers match D10-01's calculation exactly
  - Settled state only shows when remaining is zero for everyone
- **Dependencies:** D10-01

D10-04 — Couple Mode dashboard Role: [FE]

- **Objective:** Ship in-voice, per §22.
- **Flow / Brand Integration:** Room loads → FE checks `couple_mode == true` → Alters section headers (e.g., Itinerary becomes "Places to Go", Budget becomes "Budget for Two") and renders "Just the two of you?" hero text instead of standard header.
- **Tasks:**
  - Header: "Just the two of you?"; supporting: "Same OTW energy. Smaller passenger count."
  - Sections: Date Ideas, Places to Go, Budget for Two, Our Photos
- **Acceptance Criteria:**
  - A couple-mode room shows this dashboard instead of the standard one, with no duplicated backend logic
- **Dependencies:** D10-03

WEEK 3 — Photos, Location, Calendar, Hardening

(Finalizing the differentiators and UAT)

D12-01 — "The Gala Dump" screen Role: [FE]

- **Objective:** Ship in-voice, per §21.
- **Flow / Brand Integration:** User navigates to Photos section → Renders Fredoka header "The Gala Dump" → If 0 photos, shows empty state text "No memories yet. Go make some." in a `--blossom` colored UI card.
- **Tasks:**
  - CTA **Add Photos**
  - Empty state: "No memories yet. Go make some."
- **Acceptance Criteria:**
  - Matches copy exactly
  - Uploading a photo replaces the empty state with the grid
- **Dependencies:** D11-03, D11-04

D13-01 — "Who's actually OTW?" location screen Role: [FE]

- **Objective:** Ship in-voice, per §19 — keeping the actual privacy/permission info plain.
- **Flow / Brand Integration:** User views Location section → Sees "Who's actually OTW?" → Taps "Share My Location" → Native browser geolocation prompt appears → On accept, UI updates status to "On the way" and begins broadcasting.
- **Tasks:**
  - Header: "Who's actually OTW?"; CTA **Share My Location**
  - Status states: `Still at home` / `On the way` / `Almost there` / `Already there`
- **Acceptance Criteria:**
  - Location permission prompt and explanation text is clear and literal
- **Dependencies:** D12-03

D13-03 — "Want to keep your galas with you?" prompt Role: [FE]

- **Objective:** Ship in-voice, per §24 — optional, never a gate.
- **Flow / Brand Integration:** User completes primary flow → Dismissible modal slides up with `--teal` background → Header: "Want to keep your galas with you?" → Subtext: "Your current gala can still continue without an account." → User clicks "Maybe Later", modal closes immediately without friction.
- **Tasks:**
  - CTA **Create an Account**; secondary **Maybe Later**
  - Reassurance: "Your current gala can still continue without an account."
- **Acceptance Criteria:**
  - Dismissing never blocks or degrades any existing feature
- **Dependencies:** D13-02

D14-02 — "Your Gala Calendar" screen Role: [FE]

- **Objective:** Ship in-voice, per §23.
- **Flow / Brand Integration:** Authenticated user navigates to `/calendar` → Sees "Your Gala Calendar" → If no past galas, renders empty state: "Nothing on the calendar yet. Sounds like a good time to plan something."
- **Tasks:**
  - Header: "Your Gala Calendar"; empty state: "Nothing on the calendar yet."
- **Acceptance Criteria:**
  - Shown only for linked accounts; unlinked sessions never see a broken or partial version of this screen
- **Dependencies:** D14-01

UI SKILL

Design system and UI conventions for OTW ("On The Way"), a Gen-Z/Millennial Taglish group-planning app in the "Bubblegum Chaos" brand direction. Use this skill whenever building, revising, or reviewing ANY screen, mockup, landing page, component, or artifact for OTW — dashboards, wireframes, feature cards, buttons, icons, empty/error states, microcopy, etc. Trigger this even if the user doesn't say "design system" or "brand guide" — any mention of OTW, "Tara G!", "Ambagan", "Saan Tayo?", "Mga Drawing", or a request to keep OTW's UI "consistent" should load this skill first, before writing any HTML/CSS or picking colors, fonts, or copy.

OTW UI Design System — "Bubblegum Chaos"

This skill is the single source of truth for OTW's visual language. Read it in full before building or editing any OTW screen. Its job is to stop each new screen from silently drifting into generic SaaS defaults (flat white cards, grey shadows, sharp corners, stock emoji) instead of OTW's established bubbly identity.

1. Brand personality (recap)

Hyper-friendly, silly, high-energy — the app should feel like the group chat where everyone's typing at once and something spontaneous is about to happen. Chaos, but the fun, low-stakes kind. Never corporate, never quiet.

2. Color tokens

Use these exact hex values as CSS custom properties in every build — never invent new brand colors:

css
```
--rosy:       #e0417c;  /* primary CTA, primary accent */
--rosy-dark:  #c22e64;  /* CTA pressed-state shadow / gradient end */
--zest:       #f8e140;  /* highlights, badges, secondary CTA fills */
--zest-dark:  #c9b418;  /* zest button pressed-state shadow */
--blossom:    #eac7df;  /* soft card backgrounds, blob cards */
--teal:       #2f8aa3;  /* body text on light bg, active nav state, contrast borders */
--teal-dark:  #1f6376;  /* teal pressed-state shadow, dark section backgrounds, footers */
--cream:      #fff8f2;  /* neutral base / card interiors, never pure white */
```

Bubblegum Chaos specifically calls for full-bleed color per section (not a neutral page with color only in accents). Alternate whole-section backgrounds — e.g. hero = Rosy, next section =

Zest, next = Teal, cards = Blossom — rather than keeping everything on a cream/white base. If a build looks like a normal SaaS page with colored buttons on white, it has drifted off-brand — go back and make entire sections carry color.

Never introduce grey shadows (`rgba(0,0,0,.1)` SaaS-card grey). Shadows should be a solid offset in the element's own darker tone (e.g. a rosy button gets a `var(--rosy-dark)` shadow, not black/grey), which is what sells the "squishy, pressable" feel.

3. Typography

- **Display / headings / buttons / logo lockups:** `Fredoka` (weights 500/600/700) — chunky, rounded, no sharp serifs. Headlines run large (32–56px) and are set with slightly tightened letter-spacing on the biggest sizes.
- **Body / UI copy / labels:** `Nunito` (weights 600–800) — always bold-leaning; OTW has no light-weight text anywhere. Avoid thin/regular weights, they read as too corporate/quiet.
- Load both via Google Fonts (`fonts.googleapis.com` / `fonts.gstatic.com`, which are allow-listed for published pages).
- Never use a serif face, never use all-caps tracked-out labels — OTW's "shouting" is done through color and size, not letterforms.

4. Shape language

- **Buttons:** always pill-shaped (`border-radius: 999px`). No rectangular or slightly-rounded buttons anywhere, ever.
- **Cards:** two registers depending on context:
  - Simple content cards (feature cards, gala cards): generous uniform rounding, ~22–28px radius, 3px white border when sitting on a colored background.
  - "Blob" cards (mascot callouts, feature icon cards, step cards in the Bubblegum direction): asymmetric organic radius, e.g. `border-radius: 46% 54% 38% 62% / 50% 42% 58% 50%;` — vary the four values per instance so blobs don't look identical/tiled.
- **Icon containers:** circular or soft-blob, white or cream fill sitting on a colored card, icon drawn in `var(--rosy)` or `var(--teal)`.
- No sharp corners anywhere in the system. If a 90° corner appears on anything other than the phone/browser frame itself, round it.

5. Icons — no emoji, ever

OTW does not use system emoji in shipped UI (headings, badges, buttons, nav, feature cards, footers). Use hand-drawn inline SVG icons instead, stroked in brand colors, matching the mascot's rounded linework:

- **Style:** `fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"`, viewBox `0 0 24 24`, sized 22–28px in cards, larger (40–52px) for hero/mascot moments.
- **Brand "sparkle" mark:** reuse this exact diamond-star path as the recurring sparkle/highlight icon everywhere a "✨" impulse shows up (badges, footer, click-burst animations):

html
```
<svg viewBox="0 0 24 24"><path d="M12 0 L14 10 L24 12 L14 14 L12 24 L10 14 L0 12 L10 10 Z" fill="currentColor"/></svg>
```

- **Core feature icons already established** (reuse these, don't redesign per-screen):
  - *Saan Tayo? / location:* pin + dot — `<path d="M12 21s-7-7.2-7-12a7 7 0 0 1 14 0c0 4.8-7 12-7 12z"/><circle cx="12" cy="9" r="2.4"/>`
  - *Ambagan / expenses:* rounded wallet — `<rect x="3" y="7" width="18" height="12" rx="4"/><path d="M3 10.5h18"/><circle cx="16.5" cy="14.5" r="1.3" fill="currentColor" stroke="none"/>`
  - *Mga Drawing / shelved plans:* notepad — `<rect x="5" y="3" width="14" height="18" rx="4"/><path d="M9 8h6M9 12h6M9 16h4"/>`
  - *Celebration / milestone burst:* radiating sparkle — `<path d="M12 3v3M12 18v3M3 12h3M18 12h3M5.6 5.6l2 2M16.4 16.4l2 2M5.6 18.4l2-2M16.4 7.6l2-2"/><circle cx="12" cy="12" r="3" fill="currentColor" stroke="none"/>`
- When a new concept needs an icon, draw a new one in the same stroke style rather than reaching for an emoji as a placeholder — a rough on-brand SVG is always better than a correct-looking emoji.

6. Motion — "squishy and bouncy"

- **Buttons:** press state scales/translates down with a solid-color offset shadow that compresses (e.g. `box-shadow: 0 7px 0 var(--rosy-dark)` at rest → `0 2px 0 var(--rosy-dark)` + `translateY(5px)` on `:active`). This is the core "squish" signature — every primary button needs it.
- **Success sparkle-burst:** on a successful/confirming action (creating a gala, confirming a vote), spawn 5–7 small sparkle-mark SVGs that pop outward and fade (`scale(.4)→scale(1.1)`, random angle/distance, ~700ms, `ease-out`). Reuse the JS pattern already built (randomized `--tx`/`--ty` CSS vars per spawned span) rather than reinventing per screen.

- **Card hover (desktop only):** a slight lift + rotate (`translateY(-4px) rotate(-1deg)`) reads as "picked up," not a generic shadow-grow.
- Keep motion sparse and purposeful per the frontend-design skill's general guidance — the squish + sparkle are OTW's *one* signature motion pair, not a license to animate every element.

7. Microcopy & tone

- **Taglish, casual, exclamation-forward.** Talk like a friend inviting you somewhere, not a product describing a feature.
- **Established terms — always use these exact words, never generic equivalents:**

Say this	Not this
Tara G!	Create Trip / Get Started
Saan Tayo?	Location / Set Location
Ambagan	Expenses / Split Bill
Mga Drawing	Archived / Cancelled Plans

- Buttons state the action in active voice ("Start a Gala," not "Submit"); an action keeps its name through the whole flow (a "Tara G!" button leads to a gala, not a "trip").
- Empty and error states speak in-voice, not apologetically — explain what's missing and give one clear next tap, without corporate hedging ("Oops! Something went wrong" is too generic; be specific about what happened and what to do).

8. Layout conventions established so far

- **Nav:** logo mark (image or wordmark) left, text links center (hidden under ~820px), one pill CTA right.
- **Hero:** two-column on desktop (copy+CTA left, mascot/visual right), single column stacked on mobile; badge pill above the headline, oversized Fredoka headline, one primary pill CTA + one ghost/text secondary link.
- **Feature sections:** 3-card grid on desktop, single column under ~900px; one icon-blob + heading + one short line + a small colored tag per card.
- **Section rhythm for a full landing page (Bubblegum direction):** Hero (Rosy) → How-it-works (Zest) → Features (Teal, Blossom cards) → Closing CTA (Blossom) → Footer (Teal-dark). Alternate warm/cool full-bleed sections rather than repeating the same background twice in a row.

- **Mobile app screens:** bottom nav with exactly 3 large circular icon buttons (Home / Notifications / Profile), active state = solid Teal fill with a Teal-dark pressed-shadow, inactive = muted outline color. Never add a 4th or 5th nav icon or a text label under the icons — this was specified as a hard constraint (no small text links, no complicated buttons).

9. Build hygiene for published pages

- Load Google Fonts via the two allow-listed `fonts.googleapis.com` / `fonts.gstatic.com` links; no other external assets.
- Any uploaded brand asset (logo, photography) gets embedded as a `data:` URI (resized/compressed first, ~40–60KB target) so the page stays self-contained — never leave a broken external image path in a published artifact.
- Keep builds lean: one self-contained HTML file, CSS variables for every brand color (never hard-code a hex outside the `:root` block), inline SVG for every icon/diagram, no build step.

10. Quick pre-ship checklist

Before calling an OTW screen done, check:

- Every button is pill-shaped with a solid-color pressed shadow (no rectangular or grey-shadowed buttons)
- No emoji anywhere in the shipped markup — every icon is an inline SVG in the stroke style above
- Fredoka on headings/buttons, Nunito (bold-leaning) on body — no other typefaces, no thin weights
- Colors pulled only from the 8 tokens in §2, applied via CSS variables
- At least one full-bleed colored section (not just a white page with colored buttons)
- Brand terms used correctly: Tara G!, Saan Tayo?, Ambagan, Mga Drawing
- Primary CTA has the squish-press interaction; a confirming action has the sparkle-burst